

Phase 4 : Validation Automatisée de la Qualité (Great Expectations)

1. Objectifs de la validation

L'objectif de cette phase est d'établir une "Quality Gate" (Barrière de Qualité) stricte entre l'entrepôt de données nettoyées et les outils de visualisation (Superset). Nous avons choisi le framework Great Expectations (GX) pour automatiser ces tests.

L'approche repose sur la couverture des 6 Piliers de la Data Quality :

- Complétude : Vérifier que les champs obligatoires (SIRET) ne sont jamais vides.
- Unicité : Garantir qu'il n'y a pas de doublons de clés primaires.
- Exactitude & Format : Valider les formats (Regex) des Codes Postaux (5 chiffres) et Codes APE.
- Validité : S'assurer que les valeurs appartiennent à des référentiels connus (État Administratif 'A' ou 'F').
- Cohérence : Vérifier la logique métier temporelle (Date de Création <= Date de Traitement).
- Actualité & Volumétrie : S'assurer que les données sont récentes (>2000) et que le volume de données est cohérent (>1000 lignes).

2. Architecture technique mise en place

Il n'y a pas de validation manuelle, tout est intégré dans l'environnement Python/Docker, le flux de validation suit cette logique :

- On se connecte à la base PostgreSQL
- On transforme les données en PandasDataset pour fonctionner avec Great Expectations
- On applique les "Expectations" (1à règles pour les 6 piliers de la qualité cités plus haut)
- On génère un rapport de validation au format JSON et on l'affiche au format Tableau en résultat

3. Défis techniques et résolution

Nous avons eu de nombreux problèmes de dépendances, l'installation standard de Great Expectations entrainait en conflit avec les versions récentes de Pandas (2.x) et Numpy (2.0), provoquant des erreurs d'incompatibilité binaire (ValueError: numpy.dtype size changed) et des erreurs d'importation sur les modules cloud (Google/Azure) non utilisés mais requis par le framework.

Nous avons donc forcé l'utilisation de versions :

- Great Expectations V0.16.15
- Numpy < 2.0
- Installation de connecteurs fantômes (Pyspark, Boto3) pour satisfaire les exigences strictes du framework lors de l'initialisation.

4. Implémentation de la suite de tests

Nous avons programmé 10 règles principales, voici un extrait de la logique implémentée dans le Notebook :

```
# Exemple de règle de Cohérence Temporelle
# Vérifie que la création de l'entreprise est antérieure à son dernier traitement
ge_df.expect_column_pair_values_a_to_be_less_than_or_equal_to_b(
    "dateCreationEtablissement",
    "dateDernierTraitementEtablissement",
    ignore_row_if="either_value_is_missing"
)
```

On a implémenté aussi d'autres règles clés comme le matching du siret avec une règle regex simple (`^\d{14}$`) obligeant 14 chiffres consécutifs exactement, ainsi que l'obligation d'être non null.

5. Résultats et preuve de qualité

L'exécution du pipeline de validation sur la table `cleaned_stock_etablissement` (78 888 lignes) s'est finie sur un succès, et génère un fichier "validation_results.json" à chaque exécution, permettant l'historisation de la qualité.

```
Démarrage de Great Expectations...
Données chargées : 78888 lignes.
Exécution des tests de qualité...
```

```
--- RÉSULTATS ---
```

```
Statut Global : SUCCES - Données Valides
Score de Qualité : 100.0%
```

```
Rapport JSON généré : 'validation_results.json'
```

Date du rapport : 2026-02-05T13:36:25.776267+00:00
Score Global : 100.0%

	Colonne	Règle Vérifiée	Résultat
0	siret	Complétude (Non vide)	VALIDE
1	siret	Format (Regex)	VALIDE
2	siret	Unicité	VALIDE
3	libelleCommuneEtablissement	Complétude (Non vide)	VALIDE
4	codePostalEtablissement	Format (Regex)	VALIDE
5	etatAdministratifEtablissement	Validité (Liste)	VALIDE
6	activitePrincipaleEtablissement	Format (Regex)	VALIDE
7	dateDernierTraitementEtablissement	Bornes Temporelles	VALIDE
8	dates_coherentes	Validité (Liste)	VALIDE
9	Tableau entier	Bornes Temporelles	VALIDE

6. Checkpoints et Alerting

Conformément aux exigences du projet, nous avons conçu l'architecture d'automatisation des tests. Bien que l'implémentation actuelle repose sur une exécution manuelle en environnement de développement (Jupyter) pour valider les règles, la logique de Checkpoints et d'Alerting a été codée et est prête à être déployée dans l'orchestrateur.

a. Le mécanisme de “Checkpoint”

Le script génère systématiquement un fichier `validation_result.json` et calcule un score de succès. Une condition stricte a été intégrée : si le taux de succès n'est pas de 100% sur les règles critiques (Unicité du SIRET par exemple), le script lève une exception.

b. Alerting et notification

Pour l'aspect notification (Slack/Email), nous avons défini le workflow d'intégration avec Apache Airflow, bien que la connexion API réelle (Webhook Slack) n'ait pas été activée dans ce prototype pour des raisons de sécurité et de gestion des secrets.

L'architecture cible repose sur le mécanisme de Callbacks d'Airflow :

- Détection : La tâche Airflow exécutant le script de validation détecte l'exception levée par Great Expectations.
- Action sur échec : Le paramètre `on_failure_callback` du DAG déclenche une fonction d'alerte.
- Notification : Un message est envoyé sur le canal Slack `#data-quality` contenant le lien vers le rapport d'erreur.

c. Simulation d'intégration du code

```
# Logique implémentée dans le script de validation
if validation_result["success"] is False:
    # 1. On stocke la preuve de l'échec
    save_json(validation_result, "failure_report.json")

    # 2. Simulation de l'appel API Slack (Réalisable via Airflow Hooks)
    print("❌ ALERTE CRITIQUE : Envoi notification Slack...")
    # slack_hook.send(text="Échec validation Qualité SIRENE")

    # 3. Arrêt immédiat du pipeline (Fail-Fast)
    raise ValueError("Quality Gate Failed: Pipeline Stopped.")
```

Cette approche garantit que l'alerting est piloté par la donnée elle-même. Il est préférable d'arrêter le pipeline et d'alerter l'ingénieur plutôt que d'alimenter le Dashboard avec des données corrompues.

7. Conclusion de la phase

Configurer les webhooks Slack nécessitait de gérer des tokens de sécurité et des accès internet sortants qui complexifiaient l'architecture locale en plus de devoir créer un compte Slack, une communauté etc. Cependant, comme vous le voyez dans le code, la logique d'interception d'erreur est bien là. Remplacer le `print()` par un appel `slack_hook.send()` est une opération possible plus tard.

L'intégration de Great Expectations garantit que seules des données fiables alimentent les tableaux de bord. Bien que l'interface graphique (Data Docs sur le port 8080) ait été remplacée par un reporting direct via Jupyter pour des raisons d'optimisation Docker, la couverture fonctionnelle est complète et le processus est prêt pour une industrialisation via Airflow.